

Описание проекта

Описание проекта: классификация городских участков по уличным панорамам

Автор: Дмитрий Новичков, ПИК AI Lab

Задача

Всё началось с простого вопроса от городских аналитиков: как понять, что на самом деле происходит на тысячах земельных участков, где идёт стройка, где всё замерло, где территория деградирует. Спутниковый снимок такие вещи почти не показывает, а кадастровые описания часто устаревают на годы.

Идея проекта в том, чтобы взять контур участка из GeoPackage, подтянуть к нему уличные панорамы и по картинке определить состояние территории. Каждому участку присваивается один из шести классов городской среды (таксономия УТТ) и подкласс, а вместе с ними уверенность модели и прямоугольник на кадре, который показывает, на основании чего сделан вывод. В итоге аналитик получает не просто метку, а объяснение, и может быстро расставить приоритеты, куда смотреть в первую очередь.

Данные

Разметку я в основном беру из OpenStreetMap: по нему собирал панорамы, распределённые по шести классам. Сами панорамы берутся из Яндекса (равнопрямоугольные кадры примерно 7168×3584), а контуры участков, кадастр и текстовые атрибуты приходят из GPKG. Отдельно накапливается обратная связь из веб-интерфейса, где аналитик отмечает «верно/неверно», и эти отметки со временем превращаются в дополнительную обучающую выборку.

Шесть классов УТТ это `natural_areas`, `low_density_degraded`, `underused_infrastructure`, `frozen_construction`, `active_construction` и `active_urban`; у каждого есть от двух до пяти подклассов.

Отдельная история, как из панорамы получить нормальную обучающую картинку. Я не режу кадр по центру участка, а строю точки обзора вдоль граней полигона, примерно в 22 метрах от ребра, и уже оттуда делаю перспективные кропы 896×672 по азимуту съёмки. Сплит на `train` и `val` делается по объектам OSM, чтобы все сезоны одного объекта не расползались между выборками и не давали утечку. К этому моменту у меня уже собраны около 3200 объектов и порядка 13 000 кропов (примерно 10 400 в `train` и 2600 в `val`). Поверх этого я отдельно подготовил VLM-датасет: около 1100 панорам с разметкой класса и `bbox`-обоснования, на которых модель учится не просто угадывать метку, а показывать, где именно на кадре она видит признак.

Самым важным шагом оказалась не модель, а диагностика данных. Когда я прогнал аудит выборки, выяснилось, что почти у трети сэмплов азимут был нулевым: точка съёмки стояла прямо на объекте, и направление взгляда просто не определялось. Такие кадры смотрели «в никуда». Я починил это мульти-ракурсом: для проблемных случаев генерируются кропы по сторонам света, и участок всё-таки попадает в кадр.

Методы

Общая логика простая: из контура строим точки обзора, тянем панораму, режем перспективный кроп и отдаём его модели, которая возвращает класс и прямоугольник-обоснование.

Для классификации кропа я двигался от простого к сложному. Zero-shot OpenCLIP по шести классам давал около 0.42 accuracy, уже заметно выше случайного, но для продукта мало. Обычный ResNet18 в роли базовой CV-линии дотягивал до 0.71. После чистки данных и дообучения OpenCLIP точность поднялась до 0.83, а ансамбль OpenCLIP и ResNet18 дал 0.86 при macro-F1 0.84. То есть от старта до финала точность выросла примерно вдвое, и основной вклад внесла именно работа с данными, а не смена архитектуры.

Вторую и, на мой взгляд, самую интересную часть я тоже уже довёл до рабочего состояния. Обычный классификатор отвечает только «какой класс», но аналитику важнее понимать, почему. Поэтому я добавил vision-language модель, которая на одном проходе выдаёт и класс, и прямоугольник-обоснование на кадре, то есть буквально показывает, за какой кусок картинки она зацепилась. Это снимает эффект чёрного ящика: если модель ошиблась, сразу видно, что именно её сбило.

Начал я с честного замера zero-shot. Просил Qwen3-VL-8B по ширине панорамы разметить интервалы классов, и получил macro IoU около 0.34. Как быстрый ориентир без обучения это неплохо, но на смешанной городской сцене модель ленится и нарезает кадр на примерно равные вертикальные полосы, вместо того чтобы искать реальные границы застройки. Стало понятно, что для продукта нужен fine-tune.

Дальше я собрал под это отдельный VLM-датасет. К каждому кропу добавил текстовый промпт с bbox участка и целевой ответ в фиксированном формате: класс плюс координаты обоснования. Часть прямоугольников-обоснований я получил не вручную, а через weak supervision, комбинируя zero-shot предсказания, тепловые карты активаций и правки аналитиков в интерфейсе. Так удалось собрать обучающие пары без отдельной команды разметчиков.

На этом датасете я дообучил PaliGemma 3B. Архитектурно это связка визуального энкодера SigLIP-400M и языковой модели Gemma-2B, а обучал я её через LoRA (rank 16) с замороженным энкодером, так что тюнится только тонкая надстройка. Такой рецепт дал заметный скачок: macro IoU поднялся с 0.34 до 0.63, F1 до 0.76, а качество самого обоснования (evidence IoU) до 0.71. При этом одно обучение укладывается примерно в три часа на A100, поэтому цикл экспериментов получается быстрым и я могу перебирать промпты и подвыборки почти ежедневно. Отдельно радует, что геометрия участка в промпте реально помогает: когда модель заранее знает, где в кадре находится сам участок, она перестаёт путать соседние объекты и точнее локализует признак.

Вокруг моделей собран рабочий сервис на Flask: загрузка GPKG, карта на Leaflet, панорамы по граням участка, правка классов аналитиком и экспорт. ML-классификация встроена прямо в приложение, а загрузку и разворачивание панорам я написал сам (pano.py, perspective.py). Стек по мелочи: Python, PyTorch, HuggingFace, PEFT, OpenCLIP, GeoPandas, Flask и Docker.

Результаты

Если собрать всё вместе, на валидации получается такая картина:

Метрика	Значение
Accuracy, ансамбль	0.86

Метрика	Значение
Macro-F1, ансамбль	0.84
Accuracy, дообученный OpenCLIP	0.83
Macro IoU, PaliGemma	0.63
Precision / Recall / F1, PaliGemma	0.79 / 0.73 / 0.76
Evidence IoU	0.71
Доля участков в автоклассификации	84%

По классам качество ожидаемо разное: лучше всего распознаются природные территории (F1 0.88) и активная застройка (0.84), заметно тяжелее идёт малоэтажная деградирующая застройка (0.68), где визуальные признаки размытые.

Главное, что из этого выросло не только число в таблице, а работающий инструмент: веб-пилот для аналитиков с классификацией по панораме, воспроизводимый конвейер в репозитории и аудит данных с автоматическим исправлением ракурса. *Weak supervision*, то есть комбинация zero-shot и правок в интерфейсе, позволила обойтись без профессиональных разметчиков и сократила разметку примерно с 200 часов до 14.

Главный вывод для меня получился почти банальным, но выстраданным: чистые данные и геометрия участка дают больше, чем погоня за модной архитектурой. А связка GIS, компьютерного зрения и генеративных моделей реально превращает обычные уличные панорамы в рабочий инструмент городской аналитики.

Если коротко про то, что уже сделано: собран и вычищен датасет, отлажен весь конвейер от контура до кропа, обучены классификатор на кропах и отдельная VLM-модель с обоснованиями, а поверх работает веб-пилот для аналитиков с автоматической разметкой через *weak supervision*.

Дальше хочу масштабировать то, что уже работает, и в первую очередь докрутить VLM: вторая версия PaliGemma на сложных примерах, вывод подкласса и уверенности прямо в генерацию, расширение VLM-датасета через активное обучение. Важно, что модель должна остаться одна и универсальная: не отдельные сетки под зиму и лето, а единая всесезонная модель. Параллельно планирую увеличить общий датасет с 3200 до десяти с лишним тысяч объектов, перенести подход на Санкт-Петербург и Екатеринбург и вынести инференс VLM в production-API.

А дальше у меня есть более амбициозная идея. Реальные панорамы всегда упираются в потолок: их конечное число, редкие классы вроде замороженной стройки встречаются нечасто, а ручная разметка обоснований дорогая. Поэтому я хочу зайти со стороны генеративных моделей мира, которые учатся не просто рисовать картинки, а держать в голове устройство сцены: геометрию улицы, застройку, время года, освещение. Обучив такую модель на реальных панорамах с разметкой, можно генерировать синтетические города и облетать их виртуальной камерой. Раз сцену мы задаём сами, мы заранее знаем правильный ответ: и класс, и точный bbox-обоснование получаются бесплатно, прямо из параметров генерации. Это превращает пайплайн в бесконечный генератор размеченных данных, где можно осознанно штамповать именно редкие и пограничные случаи, крутить сезоны и ракурсы, дообучать модели на синтетике и калибровать их на небольшом наборе реальных панорам. Так замыкается цикл: реальные данные учат модель мира, а она производит бесконечный поток размеченных сцен, который снова улучшает распознавание. Если это заведётся, узким местом станет уже не сбор данных, а только фантазия в постановке сцен.